

Proyecto Final: Sistemas de Recomendación

álgebra Lineal Numérica

Implementación en Python/IPython

Nombre del Estudiante

7 de abril de 2026

Resumen

Este proyecto aborda el problema de sistemas de recomendación colaborativa, donde se busca predecir calificaciones de usuarios a ítems basándose en patrones de comportamiento. Se implementarán en Python métodos basados en descomposición matricial: SVD truncada con estimación previa de valores faltantes, y Alternating Least Squares (ALS) que trabaja directamente con datos observados. El proyecto utiliza el dataset MovieLens de GroupLens, un benchmark estándar. Se analizará el efecto del rango de aproximación, el condicionamiento de los subproblemas y la precisión de las predicciones.

1. Introducción y Contexto Aplicado

1.1. El problema de recomendación colaborativa

Los sistemas de recomendación son omnipresentes en plataformas como Netflix, Amazon y Spotify. La recomendación colaborativa se basa en la premisa de que usuarios con gustos similares calificarán ítems de manera similar. Formalmente, se tiene una matriz $R \in \mathbb{R}^{m \times n}$ donde:

- m = número de usuarios.
- n = número de ítems (películas, productos, canciones).
- R_{ij} = calificación del usuario i al ítem j .

La mayoría de las entradas de R son desconocidas (no todos los usuarios califican todos los ítems). El objetivo es predecir las calificaciones faltantes.

1.2. Modelo de factorización matricial

Se asume que la matriz de calificaciones tiene rango bajo $k \ll \min(m, n)$:

$$R \approx UV^T \quad (1)$$

donde:

- $U \in \mathbb{R}^{m \times k}$: matriz de factores de usuario.
- $V \in \mathbb{R}^{n \times k}$: matriz de factores de item.

La predicción para el usuario i al item j es $(UV^T)_{ij}$.

1.3. Formulación como problema de optimización

Se minimiza el error sobre las calificaciones conocidas:

$$\min_{U, V} \sum_{(i, j) \in \Omega} (R_{ij} - (UV^T)_{ij})^2 + \lambda(\|U\|_F^2 + \|V\|_F^2) \quad (2)$$

donde Ω es el conjunto de entradas conocidas y λ es un parámetro de regularización.

2. Objetivos

2.1. Objetivo General

Implementar en Python y analizar métodos numéricos para factorización matricial en sistemas de recomendación, evaluando la precisión de las predicciones y la estabilidad numérica.

2.2. Objetivos Específicos

1. Cargar y preprocesar el dataset MovieLens.
2. Implementar **SVD truncada con estimación de datos faltantes** (completación mediante medias).
3. Implementar el método **Alternating Least Squares (ALS)** desde cero.
4. Analizar el efecto del rango k en la precisión y sobreajuste.
5. Estudiar el condicionamiento de los subproblemas de mínimos cuadrados en ALS.
6. Evaluar las predicciones usando métricas estándar (RMSE, MAE).

3. Fuentes de Datos

3.1. MovieLens Dataset

El dataset MovieLens es mantenido por GroupLens Research en la Universidad de Minnesota.

3.2. Versiones disponibles

Dataset	Tamaño	Usuarios	Películas	Calificaciones
MovieLens 100K	~5 MB	943	1,682	100,000
MovieLens 1M	~6 MB	6,040	3,900	1,000,000
MovieLens 10M	~70 MB	72,000	10,000	10,000,000
MovieLens 25M	~200 MB	162,000	62,000	25,000,000

Cuadro 1: Versiones de MovieLens disponibles

3.3. Enlaces de descarga

```
1 # MovieLens 100K (recomendado para desarrollo)
2 wget http://files.grouplens.org/datasets/movielens/ml-100k.zip
3 unzip ml-100k.zip
4
5 # MovieLens 1M (para experimentos de mayor escala)
6 wget http://files.grouplens.org/datasets/movielens/ml-1m.zip
7 unzip ml-1m.zip
```

Listing 1: Enlaces para descarga directa

3.4. Código para cargar los datos

```
1 #!/usr/bin/env python3
2 """
3 M\ 'odulo para cargar y preprocesar el dataset MovieLens.
4 """
5
6 import numpy as np
7 import pandas as pd
8 from scipy.sparse import csr_matrix
9 import os
10
11 def load_movielens_100k(data_path='ml-100k'):
12     """
13     Carga el dataset MovieLens 100K.
14
15     Par\ 'ametros:
16     -----
```

```

17 data_path : str
18     Ruta al directorio con los datos
19
20 Retorna:
21 -----
22 ratings : DataFrame
23     Calificaciones con columnas user_id, item_id, rating, timestamp
24 users : DataFrame
25     Datos demogr\'aficos de usuarios
26 items : DataFrame
27     Informaci\'on de pel\'iculas
28 R_sparse : csr_matrix
29     Matriz de calificaciones en formato disperso
30 """
31 # Cargar calificaciones
32 ratings = pd.read_csv(
33     os.path.join(data_path, 'u.data'),
34     sep='\t',
35     names=['user_id', 'item_id', 'rating', 'timestamp']
36 )
37
38 # Cargar usuarios
39 users = pd.read_csv(
40     os.path.join(data_path, 'u.user'),
41     sep='|',
42     names=['user_id', 'age', 'gender', 'occupation', 'zip_code']
43 )
44
45 # Cargar items (pel\'iculas)
46 items = pd.read_csv(
47     os.path.join(data_path, 'u.item'),
48     sep='|',
49     encoding='latin-1',
50     names=['item_id', 'title', 'release_date', 'video_date',
51           'imdb_url'] + [f'genre_{i}' for i in range(19)]
52 )
53
54 # Convertir a \'indices 0-based
55 user_ids = ratings['user_id'].unique()
56 item_ids = ratings['item_id'].unique()
57
58 user_to_idx = {uid: i for i, uid in enumerate(user_ids)}
59 item_to_idx = {iid: i for i, iid in enumerate(item_ids)}
60
61 ratings['user_idx'] = ratings['user_id'].map(user_to_idx)
62 ratings['item_idx'] = ratings['item_id'].map(item_to_idx)
63
64 n_users = len(user_ids)
65 n_items = len(item_ids)
66
67 # Crear matriz dispersa
68 R_sparse = csr_matrix(
69     (ratings['rating'].values,
70      (ratings['user_idx'].values, ratings['item_idx'].values)),

```

```

71         shape=(n_users, n_items)
72     )
73
74     print(f"Dataset cargado:")
75     print(f" - Usuarios: {n_users}")
76     print(f" - Items: {n_items}")
77     print(f" - Calificaciones: {len(ratings)}")
78     print(f" - Densidad: {len(ratings) / (n_users * n_items):.4%}")
79
80     return ratings, users, items, R_sparse, user_to_idx, item_to_idx
81
82 def train_test_split(ratings, test_size=0.2, random_state=42):
83     """
84     Divide las calificaciones en entrenamiento y prueba.
85     """
86     from sklearn.model_selection import train_test_split
87
88     train, test = train_test_split(
89         ratings,
90         test_size=test_size,
91         random_state=random_state,
92         stratify=ratings['user_id']
93     )
94
95     print(f"Entrenamiento: {len(train)} calificaciones")
96     print(f"Prueba: {len(test)} calificaciones")
97
98     return train, test
99
100 def build_matrix_from_df(df, n_users, n_items):
101     """
102     Construye matriz dispersa a partir de DataFrame.
103     """
104     return csr_matrix(
105         (df['rating'].values, (df['user_idx'].values, df['item_idx'].
106         values)),
107         shape=(n_users, n_items)
108     )

```

Listing 2: Módulo de carga de datos (*src/load_data.py*)

4. Estructura del Proyecto en Python

```

1 proyecto_recomendacion/
2 ??? README.md
3 ??? requirements.txt
4 ??? data/
5 ?   ??? ml-100k/
6 ??? figures/
7 ??? src/
8 ?   ??? __init__.py
9 ?   ??? load_data.py

```

```

10 ?    ??? svd_recommender.py      # SVD con estimación de datos faltantes
11 ?    ??? als_recommender.py      # Alternating Least Squares
12 ?    ??? metrics.py
13 ?    ??? visualization.py
14 ???  scripts/
15 ?    ??? run_experiments.py
16 ?    ??? compare_rank.py
17 ?    ??? generate_recommendations.py
18 ???  tests/

```

5. Marco Teórico

5.1. SVD Truncada con Estimación de Datos Faltantes

Un enfoque para trabajar con datos incompletos es **estimar los valores faltantes** antes de aplicar SVD. Los métodos de estimación más comunes son:

- **Estimación con media global:** Se estima cada valor faltante con el promedio de todas las calificaciones disponibles.

$$\hat{R}_{ij} = \frac{1}{|\Omega|} \sum_{(p,q) \in \Omega} R_{pq} \quad (3)$$

- **Estimación con media por usuario:** Cada valor faltante se estima con el promedio de calificaciones del usuario correspondiente.

$$\hat{R}_{ij} = \frac{1}{|\Omega_i|} \sum_{j \in \Omega_i} R_{ij} \quad (4)$$

- **Estimación con media por ítem:** Cada valor faltante se estima con el promedio de calificaciones del ítem correspondiente.

$$\hat{R}_{ij} = \frac{1}{|\Omega^j|} \sum_{i \in \Omega^j} R_{ij} \quad (5)$$

Una vez estimados los valores faltantes, se obtiene una matriz densa R_{estimada} y se aplica SVD truncada:

$$R_{\text{estimada}} \approx U_k \Sigma_k V_k^T \quad (6)$$

Las predicciones para valores no observados se obtienen como $(U_k \Sigma_k^{1/2})(\Sigma_k^{1/2} V_k^T)$.

5.2. Limitaciones de la Estimación de Datos Faltantes

La estimación de datos faltantes presenta limitaciones numéricas importantes:

- **Distorsión del espectro:** Rellenar con valores constantes introduce estructura artificial que altera los valores singulares de la matriz.
- **Aumento del rango efectivo:** Una matriz de calificaciones real debería tener rango bajo, pero la estimación puede incrementar artificialmente el rango.
- **Sesgo en predicciones:** Si el método de estimación no refleja la estructura subyacente, las predicciones pueden estar sesgadas.
- **Costo computacional:** La matriz densa resultante puede ser muy grande ($m \times n$), dificultando su manejo en memoria.

5.3. Alternating Least Squares (ALS)

A diferencia del enfoque anterior, ALS trabaja directamente con los datos observados sin necesidad de estimar los valores faltantes. El problema de optimización se resuelve alternando entre fijar U y V :

5.3.1. Actualización de factores de usuario

Fijando V , para cada usuario i se resuelve:

$$\min_{\mathbf{u}_i} \sum_{j \in \Omega_i} (R_{ij} - \mathbf{u}_i^T \mathbf{v}_j)^2 + \lambda \|\mathbf{u}_i\|_2^2 \quad (7)$$

cuya solución es:

$$\mathbf{u}_i = (V_{\Omega_i}^T V_{\Omega_i} + \lambda I)^{-1} V_{\Omega_i}^T \mathbf{r}_{\Omega_i} \quad (8)$$

5.3.2. Actualización de factores de ítem

Análogamente, para cada ítem j :

$$\mathbf{v}_j = (U_{\Omega_j}^T U_{\Omega_j} + \lambda I)^{-1} U_{\Omega_j}^T \mathbf{r}_{\Omega_j} \quad (9)$$

5.4. Condicionamiento Numérico en ALS

El número de condición de la matriz $V_{\Omega_i}^T V_{\Omega_i}$ puede ser grande si el usuario ha calificado pocos ítems. La regularización λI mejora el condicionamiento:

$$\kappa(V_{\Omega_i}^T V_{\Omega_i} + \lambda I) = \frac{\sigma_{\max}^2 + \lambda}{\sigma_{\min}^2 + \lambda} \quad (10)$$

Este análisis permite estudiar cómo la regularización afecta la estabilidad numérica del método.

5.5. Métricas de Evaluación

- RMSE (Root Mean Square Error):

$$\text{RMSE} = \sqrt{\frac{1}{|\Omega_{\text{test}}|} \sum_{(i,j) \in \Omega_{\text{test}}} (R_{ij} - \hat{R}_{ij})^2} \quad (11)$$

- MAE (Mean Absolute Error):

$$\text{MAE} = \frac{1}{|\Omega_{\text{test}}|} \sum_{(i,j) \in \Omega_{\text{test}}} |R_{ij} - \hat{R}_{ij}| \quad (12)$$

6. Implementación de Métodos

6.1. SVD con Estimación de Datos Faltantes

```
1 #!/usr/bin/env python3
2 """
3 Implementaci\on de sistema de recomendaci\on basado en SVD truncada
4 con estimaci\on previa de valores faltantes.
5 """
6
7 import numpy as np
8 from scipy.sparse import csr_matrix
9
10 class SVDRecommender:
11     """
12     Recomendador basado en SVD truncada con estimaci\on de datos
13     faltantes.
14
15     Los valores faltantes se estiman utilizando estad\isticas
16     descriptivas
17     (medias) de los datos disponibles antes de aplicar SVD.
18     """
19
20     def __init__(self, n_factors=50, estimation_method='global_mean'):
21         """
22         Par\ametros:
23         -----
24         n_factors : int
25             N\umero de factores latentes (rango de aproximaci\on)
26         estimation_method : str
27             M\etodo de estimaci\on de valores faltantes:
28             - 'global_mean': media global de todas las calificaciones
29             - 'user_mean': media por usuario
30             - 'item_mean': media por item
31             - 'zero': estimar con cero
32         """
33         self.n_factors = n_factors
34         self.estimation_method = estimation_method
```



```

33     self.U = None
34     self.s = None
35     self.Vt = None
36     self.global_mean = None
37     self.user_means = None
38     self.item_means = None
39
40     def fit(self, R_train):
41         """
42         Entrena el modelo.
43
44         Parámetros:
45         -----
46         R_train : csr_matrix
47             Matriz de calificaciones de entrenamiento
48         """
49         n_users, n_items = R_train.shape
50
51         # Calcular medias segun un metodo de estimacion
52         if self.estimated_method == 'global_mean':
53             self.global_mean = R_train.data.mean()
54             estimated_value = self.global_mean
55         elif self.estimated_method == 'user_mean':
56             self.user_means = np.array(R_train.mean(axis=1)).flatten()
57             estimated_value = self.user_means.reshape(-1, 1)
58         elif self.estimated_method == 'item_mean':
59             self.item_means = np.array(R_train.mean(axis=0)).flatten()
60             estimated_value = self.item_means.reshape(1, -1)
61         else: # 'zero'
62             estimated_value = 0
63
64         # Convertir a matriz densa estimando valores faltantes
65         R_dense = R_train.toarray()
66
67         if self.estimated_method == 'global_mean':
68             missing_mask = (R_dense == 0)
69             R_dense[missing_mask] = self.global_mean
70         elif self.estimated_method == 'user_mean':
71             for i in range(n_users):
72                 missing_mask = (R_dense[i, :] == 0)
73                 R_dense[i, missing_mask] = self.user_means[i]
74         elif self.estimated_method == 'item_mean':
75             for j in range(n_items):
76                 missing_mask = (R_dense[:, j] == 0)
77                 R_dense[missing_mask, j] = self.item_means[j]
78
79         # Aplicar SVD truncada
80         U, s, Vt = np.linalg.svd(R_dense, full_matrices=False)
81
82         # Truncar
83         self.U = U[:, :self.n_factors]
84         self.s = s[:self.n_factors]
85         self.Vt = Vt[:self.n_factors, :]
86

```

```

87     def predict(self, user_idx, item_idx):
88         """
89         Predice la calificaci\on para un par (usuario, item).
90         """
91         if self.U is None:
92             raise ValueError("El modelo no ha sido entrenado")
93
94         prediction = self.U[user_idx, :] @ self.Vt[:, item_idx]
95         return np.clip(prediction, 1, 5)
96
97     def predict_all(self, user_indices=None, item_indices=None):
98         """
99         Predice calificaciones para m\ultiples pares.
100        """
101        if user_indices is None or item_indices is None:
102            return self.U @ self.Vt
103        else:
104            predictions = np.zeros(len(user_indices))
105            for i, (u, it) in enumerate(zip(user_indices, item_indices)):
106                predictions[i] = self.predict(u, it)
107            return predictions

```

Listing 3: `src/svd_recommender.py`

6.2. Alternating Least Squares (ALS)

```

1  #!/usr/bin/env python3
2  """
3  Implementaci\on de sistema de recomendaci\on basado en ALS.
4  ALS trabaja directamente con los datos observados sin necesidad
5  de estimar valores faltantes.
6  """
7
8  import numpy as np
9  from scipy.sparse import csr_matrix
10 from tqdm import tqdm
11
12 class ALSRecommender:
13     """
14     Recomendador basado en Alternating Least Squares.
15
16     A diferencia del enfoque con estimaci\on de valores faltantes,
17     ALS trabaja exclusivamente con las calificaciones observadas,
18     evitando introducir sesgos artificiales.
19     """
20
21     def __init__(self, n_factors=50, lambda_reg=0.1, n_iterations=20,
22                  verbose=True):
23         """
24         Par\ametros:
25         -----
26         n_factors : int
27             N\umero de factores latentes

```

```

28     lambda_reg : float
29         Par\'ametro de regularizaci\'on
30     n_iterations : int
31         N\'umero de iteraciones ALS
32     verbose : bool
33         Si es True, muestra progreso
34     """
35     self.n_factors = n_factors
36     self.lambda_reg = lambda_reg
37     self.n_iterations = n_iterations
38     self.verbose = verbose
39
40     self.U = None
41     self.V = None
42     self.train_errors = []
43     self.test_errors = []
44
45     def fit(self, R_train, R_test=None):
46         """
47         Entrena el modelo usando ALS.
48
49         Par\'ametros:
50         -----
51         R_train : csr_matrix
52             Matriz de calificaciones de entrenamiento
53         R_test : csr_matrix, opcional
54             Matriz de prueba para monitoreo
55         """
56         n_users, n_items = R_train.shape
57
58         # Inicializaci\'on aleatoria
59         np.random.seed(42)
60         self.U = np.random.randn(n_users, self.n_factors) * 0.1
61         self.V = np.random.randn(n_items, self.n_factors) * 0.1
62
63         # Precomputar listas de observaciones
64         user_items = [R_train[i].nonzero()[1] for i in range(n_users)]
65         user_ratings = [R_train[i].data for i in range(n_users)]
66
67         item_users = [R_train[:, j].nonzero()[0] for j in range(n_items)]
68         item_ratings = [R_train[:, j].data for j in range(n_items)]
69
70         for iteration in range(self.n_iterations):
71             if self.verbose:
72                 print(f"Iteraci\'on {iteration + 1}/{self.n_iterations}")
73
74             # Actualizar U (factores de usuario)
75             self._update_users(user_items, user_ratings)
76
77             # Actualizar V (factores de item)
78             self._update_items(item_users, item_ratings)
79
80             # Calcular error
81             train_rmse = self.compute_rmse(R_train)

```

```

82         self.train_errors.append(train_rmse)
83
84         if R_test is not None:
85             test_rmse = self.compute_rmse(R_test)
86             self.test_errors.append(test_rmse)
87             if self.verbose:
88                 print(f"  Train RMSE: {train_rmse:.4f}, Test RMSE: {
test_rmse:.4f}")
89         else:
90             if self.verbose:
91                 print(f"  Train RMSE: {train_rmse:.4f}")
92
93     def _update_users(self, user_items, user_ratings):
94         """
95         Actualiza los factores de usuario resolviendo un sistema
96         de m\inimos cuadrados regularizados para cada usuario.
97         """
98         for i in range(len(user_items)):
99             items = user_items[i]
100             ratings = user_ratings[i]
101
102             if len(items) == 0:
103                 continue
104
105             V_i = self.V[items, :]
106
107             # Matriz del sistema:  $V_i^T V_i + \lambda I$ 
108             A = V_i.T @ V_i + self.lambda_reg * np.eye(self.n_factors)
109             b = V_i.T @ ratings
110
111             self.U[i, :] = np.linalg.solve(A, b)
112
113     def _update_items(self, item_users, item_ratings):
114         """
115         Actualiza los factores de item resolviendo un sistema
116         de m\inimos cuadrados regularizados para cada item.
117         """
118         for j in range(len(item_users)):
119             users = item_users[j]
120             ratings = item_ratings[j]
121
122             if len(users) == 0:
123                 continue
124
125             U_j = self.U[users, :]
126
127             A = U_j.T @ U_j + self.lambda_reg * np.eye(self.n_factors)
128             b = U_j.T @ ratings
129
130             self.V[j, :] = np.linalg.solve(A, b)
131
132     def predict(self, user_idx, item_idx):
133         """
134         Predice la calificaci\on para un par (usuario, item).

```

```

135     """
136     prediction = self.U[user_idx, :] @ self.V[item_idx, :]
137     return np.clip(prediction, 1, 5)
138
139     def compute_rmse(self, R):
140         """
141         Calcula RMSE sobre las calificaciones conocidas.
142         """
143         rows, cols = R.nonzero()
144         predictions = np.array([self.predict(r, c) for r, c in zip(rows,
145         cols)])
146         true_values = R.data
147
148         rmse = np.sqrt(np.mean((predictions - true_values) ** 2))
149         return rmse
150
151     def get_condition_numbers(self, R_train):
152         """
153         Analiza el condicionamiento de los subproblemas en ALS.
154         \ 'util para estudiar la estabilidad num\ 'erica del m\ 'etodo.
155         """
156         n_users, n_items = R_train.shape
157         cond_numbers = []
158
159         for i in range(n_users):
160             items = R_train[i].nonzero()[1]
161             if len(items) > self.n_factors:
162                 V_i = self.V[items, :]
163                 A = V_i.T @ V_i + self.lambda_reg * np.eye(self.n_factors)
164                 s = np.linalg.svd(A, compute_uv=False)
165                 cond_numbers.append(s[0] / s[-1])
166
167         return np.array(cond_numbers)

```

Listing 4: src/als_recommender.py

6.3. Métricas de Evaluación

```

1  #!/usr/bin/env python3
2  """
3  M\ 'odulo de m\ 'etricas para evaluaci\ 'on de sistemas de recomendaci\ 'on.
4  """
5
6  import numpy as np
7
8  def rmse(predictions, true_values):
9      """Root Mean Square Error."""
10     return np.sqrt(np.mean((predictions - true_values) ** 2))
11
12  def mae(predictions, true_values):
13      """Mean Absolute Error."""
14     return np.mean(np.abs(predictions - true_values))
15

```

```

16 def precision_at_k(recommended, relevant, k):
17     """Precision@k."""
18     recommended_k = recommended[:k]
19     relevant_set = set(relevant)
20
21     if len(recommended_k) == 0:
22         return 0.0
23
24     hits = sum(1 for item in recommended_k if item in relevant_set)
25     return hits / len(recommended_k)
26
27 def recall_at_k(recommended, relevant, k):
28     """Recall@k."""
29     recommended_k = recommended[:k]
30     relevant_set = set(relevant)
31
32     if len(relevant_set) == 0:
33         return 0.0
34
35     hits = sum(1 for item in recommended_k if item in relevant_set)
36     return hits / len(relevant_set)
37
38 def ndcg_at_k(recommended, relevant, k):
39     """Normalized Discounted Cumulative Gain @ k."""
40     recommended_k = recommended[:k]
41     relevant_set = set(relevant)
42
43     dcg = 0.0
44     for i, item in enumerate(recommended_k):
45         if item in relevant_set:
46             dcg += 1.0 / np.log2(i + 2)
47
48     ideal_dcg = sum(1.0 / np.log2(i + 2) for i in range(min(k, len(
49         relevant))))
50
51     return dcg / ideal_dcg if ideal_dcg > 0 else 0.0

```

Listing 5: src/metrics.py

6.4. Script Principal de Experimentos

```

1 #!/usr/bin/env python3
2 """
3 Script principal para ejecutar experimentos de recomendaci\on.
4 Compara SVD con estimaci\on de datos faltantes vs ALS.
5 """
6
7 import numpy as np
8 import matplotlib.pyplot as plt
9 import argparse
10 import sys
11 import time
12

```

```

13 sys.path.append('.')
14 from src.load_data import load_movielens_100k, train_test_split,
    build_matrix_from_df
15 from src.svd_recommender import SVDRecommender
16 from src.als_recommender import ALSRecommender
17 from src.metrics import rmse, mae
18
19 def main():
20     parser = argparse.ArgumentParser(description='Experimentos de
    recomendaci\'on')
21     parser.add_argument('--data', type=str, default='ml-100k', help='Ruta
    a datos')
22     parser.add_argument('--n_factors', type=int, default=20, help='N\'
    umero de factores')
23     parser.add_argument('--lambda_reg', type=float, default=0.1, help='
    Regularizaci\'on')
24     parser.add_argument('--iterations', type=int, default=20, help='
    Iteraciones ALS')
25     parser.add_argument('--estimation_method', type=str, default='
    global_mean',
26                             choices=['global_mean', 'user_mean', 'item_mean',
    'zero'],
27                             help='M\'etodo de estimaci\'on de valores
    faltantes para SVD')
28
29     args = parser.parse_args()
30
31     # Cargar datos
32     print("Cargando datos...")
33     ratings, users, items, R_sparse, user_map, item_map =
    load_movielens_100k(args.data)
34
35     # Dividir en entrenamiento y prueba
36     train_df, test_df = train_test_split(ratings, test_size=0.2)
37
38     n_users = R_sparse.shape[0]
39     n_items = R_sparse.shape[1]
40
41     R_train = build_matrix_from_df(train_df, n_users, n_items)
42     R_test = build_matrix_from_df(test_df, n_users, n_items)
43
44     print(f"\nMatriz de entrenamiento: {R_train.shape}")
45     print(f"    Calificaciones: {R_train.nnz}")
46     print(f"Matriz de prueba: {R_test.shape}")
47     print(f"    Calificaciones: {R_test.nnz}")
48
49     # ===== SVD con Estimaci\'on de Datos Faltantes =====
50     print("\n" + "="*60)
51     print("SVD con Estimaci\'on de Datos Faltantes")
52     print(f"M\'etodo de estimaci\'on: {args.estimation_method}")
53     print("="*60)
54
55     svd = SVDRecommender(n_factors=args.n_factors,
56                             estimation_method=args.estimation_method)

```

```

57     start = time.time()
58     svd.fit(R_train)
59     svd_time = time.time() - start
60
61     # Evaluaci\'on
62     test_rows, test_cols = R_test.nonzero()
63     svd_preds = svd.predict_all(test_rows, test_cols)
64     svd_rmse = rmse(svd_preds, R_test.data)
65     svd_mae = mae(svd_preds, R_test.data)
66
67     print(f"Tiempo de entrenamiento: {svd_time:.2f} s")
68     print(f"RMSE: {svd_rmse:.4f}")
69     print(f"MAE: {svd_mae:.4f}")
70
71     # ===== ALS =====
72     print("\n" + "="*60)
73     print("ALS (Alternating Least Squares)")
74     print("Trabaja directamente con datos observados, sin estimaci\'on")
75     print("="*60)
76
77     als = ALSRecommender(n_factors=args.n_factors, lambda_reg=args.
78 lambda_reg,
79                        n_iterations=args.iterations, verbose=True)
80
81     start = time.time()
82     als.fit(R_train, R_test)
83     als_time = time.time() - start
84
85     als_rmse = als.compute_rmse(R_test)
86
87     print(f"\nTiempo total: {als_time:.2f} s")
88     print(f"RMSE: {als_rmse:.4f}")
89
90     # ===== An\'alisis de Condicionamiento en ALS =====
91     print("\n" + "="*60)
92     print("An\'alisis de Condicionamiento Num\'erico en ALS")
93     print("="*60)
94
95     cond_numbers = als.get_condition_numbers(R_train)
96     print(f"N\'umero de condici\'on promedio: {np.mean(cond_numbers):.2e}"
97 )
98     print(f"N\'umero de condici\'on m\'aximo: {np.max(cond_numbers):.2e}")
99     print(f"N\'umero de condici\'on m\'inimo: {np.min(cond_numbers):.2e}")
100
101     # ===== Visualizaci\'on de Resultados =====
102     fig, axes = plt.subplots(2, 2, figsize=(14, 10))
103
104     # 1. Comparaci\'on de errores
105     methods = ['SVD\n(estimaci\'on)', 'ALS']
106     rmse_values = [svd_rmse, als_rmse]
107
108     axes[0, 0].bar(methods, rmse_values, color=['steelblue', 'darkorange'
109 ])

```



```

108 axes[0, 0].set_ylabel('RMSE')
109 axes[0, 0].set_title('Comparaci\on de Error (RMSE)')
110 axes[0, 0].grid(True, alpha=0.3)
111
112 # 2. Convergencia ALS
113 axes[0, 1].plot(als.train_errors, 'b-', linewidth=2, label='
Entrenamiento')
114 axes[0, 1].plot(als.test_errors, 'r-', linewidth=2, label='Prueba')
115 axes[0, 1].set_xlabel('Iteraci\on')
116 axes[0, 1].set_ylabel('RMSE')
117 axes[0, 1].set_title('Convergencia de ALS')
118 axes[0, 1].legend()
119 axes[0, 1].grid(True, alpha=0.3)
120
121 # 3. Distribuci\on de n\umeros de condici\on en ALS
122 axes[1, 0].hist(cond_numbers, bins=50, color='darkorange', alpha=0.7)
123 axes[1, 0].set_xlabel('N\umero de condici\on')
124 axes[1, 0].set_ylabel('Frecuencia')
125 axes[1, 0].set_title('Condicionamiento de subproblemas en ALS')
126 axes[1, 0].set_xscale('log')
127 axes[1, 0].grid(True, alpha=0.3)
128
129 # 4. Tiempos de ejecuci\on
130 times = [svd_time, als_time]
131 axes[1, 1].bar(methods, times, color=['steelblue', 'darkorange'])
132 axes[1, 1].set_ylabel('Tiempo (s)')
133 axes[1, 1].set_title('Tiempo de Entrenamiento')
134 axes[1, 1].grid(True, alpha=0.3)
135
136 plt.tight_layout()
137 plt.savefig('figures/experiment_results.png', dpi=150)
138 plt.show()
139
140 # Guardar resultados
141 results = {
142     'svd': {'rmse': svd_rmse, 'mae': svd_mae, 'time': svd_time},
143     'als': {'rmse': als_rmse, 'time': als_time,
144            'condition_numbers': cond_numbers.tolist()},
145     'als_history': {'train': als.train_errors, 'test': als.test_errors
146 }
147
148 np.save('data/results.npy', results)
149 print("\nResultados guardados en data/results.npy")
150
151 if __name__ == "__main__":
152     main()

```

Listing 6: scripts/run_experiments.py

7. Requisitos de Instalaci3n

```
1 numpy>=1.20.0
2 scipy>=1.7.0
3 pandas>=1.3.0
4 matplotlib>=3.4.0
5 scikit-learn>=0.24.0
6 tqdm>=4.62.0
```

Listing 7: requirements.txt

8. Cronograma Sugerido

Semana	Actividad
1	Configuración, carga de datos y exploración
2	Implementación de SVD con estimación de datos faltantes
3	Implementación de ALS
4	Experimentos con diferentes parámetros (k , λ)
5	Análisis de condicionamiento y convergencia
6	Generación de recomendaciones, documentación y entrega

Cuadro 2: Cronograma de 6 semanas

- **Implementación** (30 %): ALS desde cero, SVD con estimación de datos faltantes.
- **Experimentación** (25 %): Variación sistemática de parámetros.
- **Análisis numérico** (20 %): Condicionamiento de subproblemas, convergencia.
- **Interpretación** (15 %): Discusión de resultados y comparación entre métodos.
- **Código y documentación** (10 %): Modularidad, docstrings, README.

9. Referencias

Referencias

- [1] F. Maxwell Harper y Joseph A. Konstan, *The MovieLens Datasets: History and Context*, ACM Transactions on Interactive Intelligent Systems, 2015.
- [2] Yehuda Koren, Robert Bell y Chris Volinsky, *Matrix Factorization Techniques for Recommender Systems*, IEEE Computer, 2009.
- [3] R. Gemulla et al., *Large-scale matrix factorization with distributed stochastic gradient descent*, KDD, 2011.
- [4] Biswa N. Datta, *Numerical Linear Algebra and Applications*, SIAM, 2nd Edition, 2010.